

Surviving Anthropic API Rate Limits in a PydanticAI Multi-Agent Pipeline

Lessons learned the hard way: transient errors, root causes, and architectural fixes

April 2026 · Stack: Python 3.14 · pydantic-ai v1.79.0 · claude-sonnet-4-6 / claude-opus-4-6

Compiled by: Douglas Mun with AI assistance.

Disclaimer

This document is a practitioner's lessons-learned record, not official documentation. It reflects the authors' direct experience debugging a specific production system during a specific period. Several important caveats apply.

Topic	Caveat
API behaviour	The connection-reset mechanism described in the Root Cause section — where the Anthropic API drops the TCP connection without returning an HTTP status code — is based on observed behaviour, not published documentation. Anthropic does not publicly document a named streaming-connection rate limit category. The inference that such a limit exists is consistent with the documented acceleration limits and 529 overloaded error classes, and has been independently observed by other developers, but it remains an empirical conclusion rather than a guaranteed platform specification. This behaviour may change without notice.
API changes	All PydanticAI code examples were verified against the live API at <code>ai.pydantic.dev</code> as of April 2026. PydanticAI is under active development and releases breaking changes between minor versions. Key identifiers to re-verify before use: <code>output_type=</code> (was <code>result_type=</code> before v1), <code>result.output</code> (was <code>result.data</code>), and <code>RunUsage</code> field names. Always check the official changelog before adopting code from this document into a new project.
Session counts	The session counts cited throughout (31–42 before, ~4 after) reflect the specific pipeline described in this paper. Your pipeline will have different counts depending on the number of agents, the number of tools each agent calls, and whether the LLM decides to call multiple tools per invocation. The architectural principles generalise; the specific numbers do not.
Third-party tools	LangGraph, LlamaIndex, Logfire, and FastAPI are documented based on their public APIs and documentation as of April 2026. LlamaIndex Workflows retry policy (issue #20403) was reported broken as of December 2025; verify the fix status before using it in production. All third-party libraries are subject to their own versioning and deprecation policies.
No warranty	The architectural patterns and code examples in this document are provided as-is for educational and reference purposes. They have not been independently audited for correctness, security, or fitness for any particular purpose. Always validate against your own requirements, load profile, and API tier before deploying to production.

Executive Summary

The problem

A production PydanticAI pipeline with eight LLM agents was failing mid-run with `httpcore.ReadError: Connection reset by peer` — on macOS, or `Errno 104` on Linux. The error looked like network instability. It was not. The root cause was a pipeline that consumed 31 to 42 Anthropic API streaming sessions per run, crossing an undocumented threshold at which the API drops the TCP connection rather than returning a standard HTTP error code.

Why it was hard to diagnose

The error message is indistinguishable from a genuine network failure. The Anthropic API does not return HTTP 429 when this limit is hit — it silently resets the TCP connection. Standard debugging paths (adding retries, lowering token limits, tightening timeouts) all failed because they addressed symptoms rather than the cause. Meanwhile, a concurrent PydanticAI agent singleton bug caused state corruption on reuse, producing different error messages on different runs and making the root cause even harder to isolate.

The fix

The solution was architectural, not operational. Every pipeline stage was audited against a single question: is this LLM reasoning, or is it routing? Stages that were mechanically calling tools with no LLM judgment were converted to direct Python subprocess calls. The three adjacent reasoning stages (ANALYZE, CORRELATE, VALIDATE) were merged into one agent with a richer system prompt. External data needed by the reasoning stage was pre-fetched in Python — zero API sessions — and bundled into a single prompt. The result: from eight LLM agents to two, and from 31–42 streaming sessions to approximately four per run.

Before	After
8 LLM agents 31–42 API sessions / run <i>Fails under load</i>	2 LLM agents + 5 Python stages ~4 API sessions / run (≈90% reduction) <i>Stable in production</i>

What this paper covers

The main body documents the five failed attempts before the architectural fix, the reasoning behind each decision, the pre-fetch and single-agent pattern in full runnable code, and eight PydanticAI patterns that helped. The appendices provide a complete PydanticAI practical reference guide, an extended Logfire observability playbook, a LangGraph integration guide covering durable checkpointing and per-node retry policies, and a framework comparison across PydanticAI, LangChain, and LlamaIndex.

The single most actionable insight

 Before writing any code, draw your pipeline's session budget. Count exactly how many Anthropic API streaming sessions each run will consume — one per `agent.run()` call, multiplied by the number of tool calls each agent makes. If that number surprises you, the architecture needs rethinking before the code does.

Who This Document Is For

You are building a multi-agent LLM pipeline with PydanticAI. Multiple AI agents run in sequence, each using tools to call external APIs, each streaming responses. It works in development. Then you run it against a real workload and — somewhere in the middle — the connection drops:

```
Transient error (attempt 1/3): Connection error
httpcore.ReadError: [Errno 54] Connection reset by peer

Transient error (attempt 2/3): Connection error
httpcore.ReadError: [Errno 54] Connection reset by peer
```

You retry. It fails again. You add backoff. It still fails. The failure point shifts each run — sometimes early in the pipeline, sometimes near the end — so it doesn't look deterministic. You suspect your network, your retry logic, a flaky API. You spend days on it.

This document is the record of what we tried, what failed, and what the problem actually was — so you can skip straight to the answer.

 *Spoiler: it was rate limiting, disguised as a network error. And the only fix was architectural.*

The Core Mental Model: What One Agent Call Actually Costs

Before describing what went wrong, it helps to understand what a single `agent.run()` call actually does on the wire when the agent has tools registered.

Most developers think of it as: one call in, one response out. The reality is very different:

```
await agent.run(prompt)

→ POST /v1/messages [streaming] ← agent decides to call a tool
  ← response: tool_use: fetch_primary_data('record_123')

  [Python subprocess runs fetch_primary_data.py]

→ POST /v1/messages [streaming] ← agent gets tool result, decides
again
  ← response: tool_use: fetch_secondary_data('record_123')

  [Python subprocess runs fetch_secondary_data.py]

→ POST /v1/messages [streaming] ← agent produces final answer
  ← response: text (structured JSON)

Total: 3 API streaming sessions for one agent.run() with 2 tool calls.
```

Every tool call is a separate streaming HTTP request. An agent calling N tools produces N+1 API sessions. This is not a PydanticAI quirk — it is how the tool-use protocol works across all LLM APIs. The LLM cannot execute a tool itself; it signals the request, your code executes it, and you send the result back in a new request.

Agent	Typical Tool Calls	API Sessions Generated	Notes
INGEST	2	~3	Classify input type, then execute source query based on that decision
COLLECT	4	~5	Data probing + record fetch
ENRICH	3–4	~4–5	External API enrichment
ANALYZE	1	~2	Pattern analysis
CORRELATE	10–20	~11–21	One reference lookup per result!
VALIDATE	1	~2	Confidence scoring + false positive filter
REPORT	1	~2	Write report files
TRANSFORM	1	~2	Data export
TOTAL (8 agents)	~23–34	~31–42 per run	More with multiple inputs

For a pipeline run with 5 inputs and 10 results each, CORRELATE alone could generate 100+ API sessions. The pipeline was firing all of these in rapid succession from a single API key.

Root Cause: Rate Limiting Disguised as a Network Error

The Anthropic API publishes rate limits in terms of tokens per minute and requests per minute. What is not publicly documented — but what the evidence strongly suggests — is a separate limit on the rate of concurrent and sequential streaming connections per API key. This is the limit that kills multi-agent pipelines, and it is not what developers are warned about.

This observation is consistent with two things Anthropic does document. First, the API docs note that sharp increases in usage can trigger acceleration limits, causing 429 errors and recommending gradual ramp-up and consistent usage patterns. Second, 529 overloaded errors are acknowledged as a separate class that occurs under high traffic. The TCP RST behaviour described below — producing a connection-reset error rather than a clean HTTP status code — appears to be how these capacity limits surface specifically for rapid sequential streaming requests. The pattern has been independently observed and reported by developers across Anthropic's community channels and support forum.

When this threshold is crossed, the server does not return a clean HTTP 429 Too Many Requests. Based on observed behaviour, it drops the TCP connection with a RST packet. The Python HTTP stack (httpcore) interprets a connection reset as a network error — `ReadError: [Errno 54] Connection reset by peer on macOS` or `ReadError: [Errno 104] Connection reset by peer on Linux` — which is indistinguishable from a genuine network partition.

Why the failure was so hard to diagnose

- The error message describes a network symptom, not a quota cause
- The failure point varied because the limit is measured over a sliding time window
- Running agents in parallel doubled the simultaneous session count, occasionally pushing over the limit at different points depending on timing

⚡ *The critical insight: this error is not random. Any pipeline that exceeds the connection threshold will fail. The variance in where it fails is a time-window artefact. The only fix is to send fewer sessions.*

What We Tried — and Why It Failed

Attempt 1 — Retry with Exponential Backoff

```
from httpx import ConnectError, ReadTimeout, RemoteProtocolError
from anthropic import APIStatusError

def _is_retryable(exc: Exception) -> bool:
    """Return True for errors that may resolve on retry.

    WARNING: connection resets (Errno 54 / 104) look retryable but are
    often session-rate limiting in disguise. If you see repeated resets,
    count your total API sessions BEFORE assuming this is network noise.
    """
    # Genuine transient network blips — safe to retry
    if isinstance(exc, (ConnectError, ReadTimeout, RemoteProtocolError)):
        return True
    # HTTP 500 (Anthropic internal error) or 529 (overloaded) — retry
    once
    if isinstance(exc, APIStatusError) and exc.status_code in (500, 529):
        return True
    # Connection reset — MAY be rate limiting; retry cautiously
    if 'connection reset' in str(exc).lower():
        return True
    return False

async def _run_with_retry(
    agent: Agent,
    prompt: str,
    deps: Any,
    max_attempts: int = 3,
) -> Any:
    """Wrap agent.run() with exponential backoff.

    Handles genuine transient failures — NOT rate limiting.
    Each retry fires a new streaming session, compounding the
    problem if the root cause is session-rate exhaustion.
    """
    for attempt in range(1, max_attempts + 1):
        try:
            return await agent.run(prompt, deps=deps)
```

```

except Exception as e:
    if attempt == max_attempts or not _is_retryable(e):
        raise
    delay = BASE_DELAY * (2 ** (attempt - 1)) + random.uniform(0,
5)
    await asyncio.sleep(delay)

```

Why it failed: Retrying a rate-limited connection doesn't help — it makes things worse. Each retry fires another streaming session into the same rate-limited window. Each failed attempt also burned tokens (the request was partially processed before the TCP RST), costing money with no benefit. Three retries with delays of ~5s, ~12s, and ~22s wasted 40 seconds and accumulated costs before the pipeline aborted.

 *The lesson: retry logic addresses transient failures (brief network partition, upstream restart). Rate limiting is not transient — it is a hard capacity constraint. You cannot retry your way out of a quota. If you see connection resets from an LLM API, count your API sessions before touching your retry logic.*

Attempt 2 — Reducing max_tokens per Agent

We lowered `max_tokens` from 16000 to 4000 on several agents, reasoning that shorter responses would complete faster and be less likely to get cut off.

Why it backfired: `max_tokens` controls a single response's length, not the number of API sessions. The rate limit was on session count, not token volume. Reducing `max_tokens` did nothing for the rate limit — and caused agents that output large JSON arrays to have their responses truncated mid-object. The JSON parser threw exceptions and introduced a new failure mode.

 *The lesson: for tool-using agents, the metric that matters for rate limits is sessions per run, not tokens per call. Never conflate the two. Set `max_tokens` high enough that structured outputs complete fully — then address session count as a separate problem.*

Attempt 3 — Explicit httpx Timeouts and max_retries=0

While debugging we added explicit timeout configuration and disabled the SDK's built-in retry logic, which would otherwise silently duplicate failed requests.

```

import httpx
from anthropic import AsyncAnthropic

client = AsyncAnthropic(
    api_key=os.environ.get('ANTHROPIC_API_KEY'),
    # Disable SDK auto-retry — we own all retry logic.
    # Without this, the SDK silently fires duplicate requests
    # on failure, inflating your session count.
    max_retries=0,
    timeout=httpx.Timeout(
        connect=15.0, # max time to establish the TCP connection

```

```

        read=300.0,      # max time to wait for streaming response data
        write=30.0,      # max time to send the request body
        pool=15.0,       # max time to acquire a connection from the pool
    ),
)
# Note: the official Anthropic SDKs set TCP socket keep-alive
# automatically
# on non-streaming requests, reducing idle connection drops on unreliable
# networks. When building a direct client as above, this is handled for
# you by the underlying httpx transport.

```

Effect: This was genuine hygiene, not a cure. `max_retries=0` stopped the SDK from silently duplicating requests on failure — which would have compounded the session count. Explicit timeouts prevented indefinite hangs on a dead connection. Good to have. Didn't reduce session count at all.

Note: PydanticAI v1 now ships an official retry module — `pydantic_ai.retries` — that provides `AsyncTenacityTransport` for retrying individual HTTP requests with exponential backoff and `Retry-After` header support. See Pattern 8 later in this paper for how to wire it with `AnthropicProvider`. For the pipeline problem described here, transport-level retries are better than agent-level retries — but neither addresses the root cause of too many sessions.

 *The lesson: always configure explicit timeouts and disable SDK retries when you own retry logic. But file this under baseline hygiene, not a solution to rate limiting.*

Attempt 4 — Efficiency Rules in System Prompts

```

## EFFICIENCY RULES — IMPORTANT
- Make the MINIMUM number of tool calls needed.
- Per input: call fetch_primary_data ONCE, then classify_record ONCE.
That is all.
- Skip expensive probes unless the input is high-priority.
- Do NOT retry tools on failure — record the error and move on
immediately.

```

Effect: Real but limited. Before this change, one agent was calling the same tool 2–3 times per input record (trying different query modes individually). After the prompt change: once. That agent's session count dropped from ~6 to ~3.

Why it wasn't enough: With 8 agents each requiring a minimum of one tool call, the pipeline floor was still 8+ sessions. You cannot prompt-engineer your way below the number of agents. Prompt efficiency is a multiplier on architecture — if the architecture is wrong, no prompt will fix it.

 *The lesson: prompting reduces waste within an agent. Architecture determines the ceiling. Work on both, but only one addresses the root problem.*

Attempt 5 — Converting One Agent to Direct Python

After the prompt work, we looked at one external-API enrichment agent and asked: what is the LLM actually doing here?

The answer: nothing. It was mechanically calling `fetch_primary_data`, receiving structured JSON, and passing it downstream unchanged. The LLM added no reasoning or interpretation. It was routing.

We replaced it with direct Python:

```
# Before: LLM agent (3-5 API sessions)
result = await _run_with_retry(enrich_agent, prompt, deps=state)

# After: direct subprocess calls (0 API sessions)
primary_raw = await run_tool('fetch_primary_data.py', 'lookup',
record_id)
secondary_raw = await run_tool('fetch_secondary_data.py', 'lookup',
record_id)
```

Result: Output quality was unchanged — because there was never any reasoning happening. This was the pivotal moment. We had been treating 'LLM agent' as the default architecture and Python as the fallback. We had it exactly backwards.

✓ *The lesson: an agent that only calls tools and passes results through is a router, not a reasoner. Routing belongs in Python. If removing the LLM from a stage produces identical output, the LLM was never needed there.*

A Concurrent Issue: The PydanticAI Singleton Bug

While debugging the connection resets, a second unrelated problem surfaced and made the investigation significantly harder: PydanticAI Agent objects accumulate internal state after the first streaming `.run()` call. Reusing the same instance on a second run — the natural pattern when agents are defined at module level — causes silent failures with no clear error message pointing to the cause.

This is why the connection reset failures were inconsistent even within the same run: the singleton bug could corrupt agent state independently of the rate-limiting issue, producing failures that looked like different problems each time. Once both issues existed simultaneously, no single fix resolved all symptoms. The singleton bug masked itself as a network problem and vice versa.

```
# BAD: module-level singleton — constructed once, reused across all
calls.
# After the first streaming .run(), PydanticAI retains graph state.
# The second invocation inherits that dirty state → silent failure.

# System prompt constants must be static strings (see run_pipeline
section).
# Defining them here so this code block is self-contained.
```

```

INGEST_SYSTEM_PROMPT = 'You are an ingestion agent. Classify input
type...'

ingest_agent = Agent(model='anthropic:claude-sonnet-4-6',
                     system_prompt=INGEST_SYSTEM_PROMPT)
@ingest_agent.tool
async def source_query(ctx, record_id): ...

# Works on first call. Breaks silently on the second.

# CORRECT: factory function – fresh instance, fresh connection pool,
every call.
def make_ingest_agent() -> Agent[PipelineState, IngestResult]:
    """Return a fresh INGEST agent for one pipeline invocation.

    Always call this function – never reuse an existing instance.
    A new Agent also creates a fresh httpx connection pool, avoiding
    socket reuse from a pool holding a connection closed by the server
    during a prior rate-limit event.
    """
    agent: Agent[PipelineState, IngestResult] = Agent(
        model='anthropic:claude-sonnet-4-6',
        output_type=IngestResult,
        system_prompt=INGEST_SYSTEM_PROMPT,
        defer_model_check=True,
        model_settings=ModelSettings(max_tokens=4000),
    )
    @agent.tool
    async def source_query(ctx: RunContext[PipelineState], record_id:
str) -> str:
        """Fetch the canonical record. Docstring is read by the LLM as
guidance."""
        return await run_tool('source_query.py', 'fetch', record_id)
    return agent

```

✓ **Rule:** in PydanticAI, always use factory functions. Treat agent instances as single-use objects. The pattern is especially important in pipelines: a reused agent + a rate-limit-closed socket is two bugs that look like one intermittent network problem.

What Finally Worked: Redesigning the Architecture

The five attempts above were all attempts to fix a symptom. The root problem was a design error: we built every pipeline stage as an LLM agent, regardless of whether that stage required LLM reasoning.

The fix required stepping back and asking a question we should have asked at the start:

🔍 *For each stage in the pipeline: is the LLM reasoning, or is it routing?*

Stage	What the LLM was actually doing	Needs LLM?	Replacement
INGEST	Classify ambiguous input type; decide query strategy; execute the resulting source query (2 tool calls: classify, then fetch)	Yes	Keep as LLM agent
COLLECT	Call tool_a, then tool_b, then tool_c, in a fixed predetermined order	No	Direct Python
ENRICH	Call an external API and pass the structured result downstream unchanged	No	Direct Python
ANALYZE	Interpret unstructured response data; infer implicit attributes not explicitly stated in the raw output	Yes	Merge into REASON
CORRELATE	Reason across multiple reference sources to assess applicability; weigh conflicting signals	Yes	Merge into REASON
VALIDATE	Apply judgment-based confidence scoring; distinguish genuine anomalies from noise using contextual reasoning	Yes	Merge into REASON
REPORT	Format structured results into a markdown template using deterministic rules	No	Direct Python
TRANSFORM	Build graph nodes and edges from typed state fields — pure data mapping	No	Direct Python

Five of eight agents did not need to be LLM agents. The remaining three — ANALYZE, CORRELATE, and VALIDATE — all operated on the same pre-collected data and performed one continuous reasoning task. They were merged into a single REASON agent with no tools.

Before (8 LLM Agents · 31–42 sessions/run)	After (2 LLM Agents · ~4 sessions/run)
INGEST (LLM + tools, ~3 sessions)	INGEST (LLM + tools, ~3 sessions) ← unchanged
COLLECT (LLM + tools, ~5 sessions)	COLLECT (direct Python, 0 sessions)
ENRICH (LLM + tools, ~4 sessions)	ENRICH (direct Python, 0 sessions)
ANALYZE (LLM + tools, ~2 sessions)	REASON (LLM, NO TOOLS, 1 session)
CORRELATE (LLM + tools, ~11–21 sessions)	<i>merged into REASON above</i>
VALIDATE (LLM + tools, ~2 sessions)	<i>merged into REASON above</i>
REPORT (LLM + tools, ~2 sessions)	REPORT (direct Python, 0 sessions)
TRANSFORM (LLM + tools, ~2 sessions)	TRANSFORM (direct Python, 0 sessions)
Total: 31–42 sessions/run	Total: ~4 sessions/run (~90% reduction)

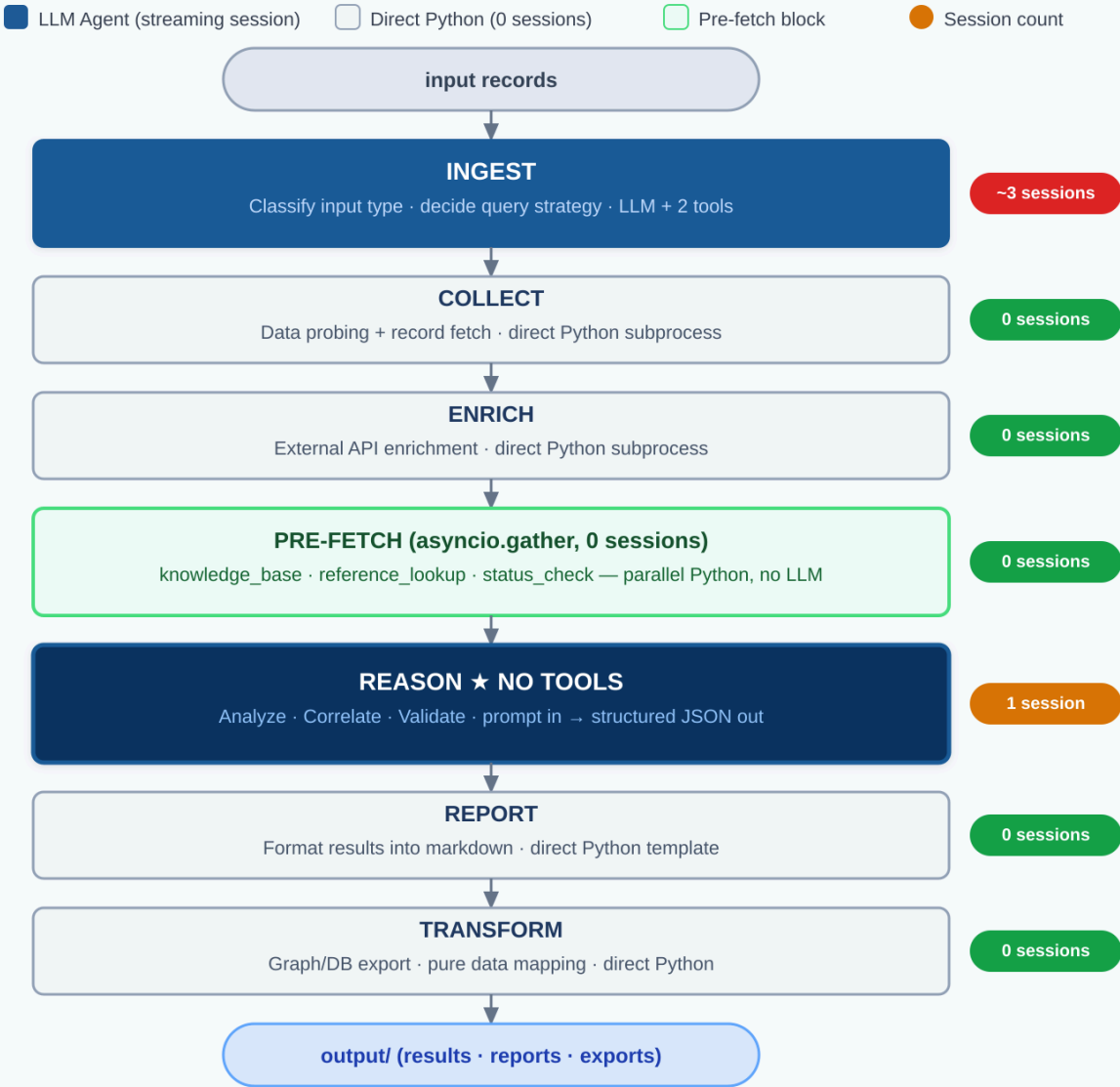
The redesigned pipeline completed end-to-end without a single connection error on the first attempt.

Pipeline Architecture Diagram

The diagram below shows the redesigned architecture. Blue boxes are LLM agents (streaming API sessions). Grey boxes run in Python with zero API sessions. Green is the pre-fetch block. Session counts are shown on the right.

Redesigned Multi-Agent Pipeline Architecture

After architectural fix · 2 LLM agents · ~4 API sessions per run (reduced from 31–42)



API Session Budget

- **Before:** 31–42 sessions/run · 8 LLM agents, all with tool calls · fails under load
- **After:** ~4 sessions/run · 2 LLM agents, 5 Python stages · ~90% reduction, stable

Figure 1 — Redesigned pipeline: 2 LLM agents + 5 Python stages ≈ 90% reduction in API sessions

The Key Innovation: The Pre-Fetch + Single-Agent Pattern

The most impactful change was replacing three tool-heavy agents with a single REASON agent that has no tools registered. The original three agents each gathered data via tool calls during their runs — combined, up to 25+ API sessions for one interconnected analysis task.

The REASON pattern inverts the approach:

Step 1 — Pre-fetch all external data in Python (zero API sessions)

Subprocess calls to external APIs or local tools consume **zero** LLM API sessions. They are ordinary Python I/O — HTTP requests, file reads, database queries. Only `agent.run()` opens a streaming session with the LLM API. By running all data collection in Python before the agent is invoked, you eliminate the tool-call back-and-forth entirely.

The helper below shows the pattern used throughout this pipeline:

```
import asyncio
from typing import Any

async def run_tool(script: str, command: str, *args: str) -> str:
    """Run an external data-collection script as a subprocess.

    Subprocess calls consume ZERO LLM API sessions — they are
    ordinary Python I/O (HTTP requests, DB queries, file reads).
    Only agent.run() opens a streaming LLM session.

    Returns stdout as a string (typically JSON).
    Raises RuntimeError if the script exits with a non-zero code.
    """
    proc = await asyncio.create_subprocess_exec(
        'python3', f'tools/{script}', command, *args,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    stdout, stderr = await proc.communicate()
    if proc.returncode != 0:
        raise RuntimeError(f'{script} {command} failed:
{stderr.decode()}')
    return stdout.decode()

# Use asyncio.gather() to run pre-fetch calls concurrently.
# Each call is a subprocess — they run in parallel with no LLM session
cost.
kb_data, ref_advisories, status_data = await asyncio.gather(
    run_tool('knowledge_base.py', 'lookup', record_id),
    run_tool('reference_lookup.py', 'query', item_id),
    run_tool('status_check.py', 'get', item_id),
)
```

Step 2 — Bundle all data into one prompt and call the LLM once (1 API session)

```
# Build a single rich prompt containing everything the LLM needs.
# The model sees the full picture in one shot — no iterative querying.
prompt = f'''
Collection results: {json.dumps(collect_data)}
Enrichment results: {json.dumps(enrich_data)}
Knowledge base data: {json.dumps(kb_data)}
Reference advisories: {json.dumps(ref_advisories)}
Status data: {json.dumps(status_data)}

Perform pattern analysis, reference correlation, and validation.
Return structured JSON matching the AnalysisResult schema.
'''

# Exactly ONE streaming session — the agent has no tools registered,
# so it cannot initiate any back-and-forth with the API.
result = await reason_agent.run(prompt)
```

Step 3 — Parse the structured JSON and write to DB in Python (zero API sessions)

The agent definition — no tools registered

```
import httpx
from pydantic import BaseModel, Field
from pydantic_ai import Agent
from pydantic_ai.models.anthropic import AnthropicModel
from pydantic_ai.settings import ModelSettings
from anthropic import AsyncAnthropic

class ResultItem(BaseModel):
    """A single validated result from the reasoning pass."""
    record_id: str
    label: str
    confidence: float = Field(ge=0.0, le=1.0)
    notes: str | None = None

class AnalysisResult(BaseModel):
    """Schema for the REASON agent's output.

    PydanticAI validates this after the LLM responds.
    If validation fails, it retries up to `retries` times.
    Set max_tokens large enough that the JSON is never truncated.
    """
    items: list[ResultItem]
    confidence: float = Field(ge=0.0, le=1.0)
    requires_manual_review: bool
    summary: str

def make_reason_agent() -> Agent[None, AnalysisResult]:
    """Factory: return a fresh REASON agent for one pipeline invocation.
```

No tools are registered – this is deliberate. Without tools, the agent cannot initiate any tool-use back-and-forth. The conversation is exactly two turns: prompt in → structured JSON out. Result: exactly 1 streaming API session, regardless of data volume.

Opus is used here because this agent must reason across many data sources simultaneously. In practice, Sonnet produced unreliable cross-referencing on large result sets for this specific task; your mileage may vary – benchmark both before committing.

```
"""
# Inline the model – no hidden make_opus_model() helper needed.
model = AnthropicModel(
    'claude-opus-4-6',
    anthropic_client=AsyncAnthropic(
        max_retries=0,          # we own all retry logic
        timeout=httpx.Timeout(read=180.0, connect=15.0),
    ),
)
return Agent(
    model=model,
    output_type=AnalysisResult,      # schema enforced
    automatically
    system_prompt=REASON_SYSTEM_PROMPT,
    defer_model_check=True,          # skip API ping at
    construction
    retries=2,                       # retry budget if validation
    fails
    model_settings=ModelSettings(
        temperature=0.0,           # deterministic structured
    output
        max_tokens=16000,          # must fit full JSON – never
    truncate
        timeout=120.0,             # reasoning can take 60-90s
    ),
    # NO @agent.tool decorators → exactly 1 streaming session
)
```

✓ *Why no registered tools is the key: without tools, the agent cannot initiate any back-and-forth. The conversation is exactly two turns: the prompt goes in, the structured JSON comes out. One streaming session. Period.*

Trade-offs to accept

- The LLM must reason about everything simultaneously rather than iteratively querying for more data
- **Use a capable model** — Opus 4.6, not Sonnet — for complex multi-step reasoning
- **Set `max_tokens=16000`** to fit a full structured JSON output with many items
- All data must be pre-fetched before the LLM call — Python subprocess calls never hit LLM rate limits

The reasoning quality was preserved — and arguably improved: the model could see all data simultaneously rather than reasoning from partial context across three separate calls.

PydanticAI Patterns That Helped

The seven patterns below are the specific PydanticAI features we reached for while solving the problems described in this paper. Each is presented in the context of why it mattered for the pipeline. For full reference coverage — including dependency injection, multi-turn conversation, the router pattern, input validation, union output types, result_validators, FastAPI integration, Logfire, and a framework comparison — see Appendix A.

1. ModelRetry — Recoverable Tool Errors

When a tool cannot satisfy the LLM's request, raise `ModelRetry` instead of a plain exception. The framework appends your message to the conversation as a `tool_result` and the LLM sees it in context — allowing it to correct course and try again. A plain `ValueError` crashes the entire run and loses all work done so far.

```
from pydantic_ai import Agent, RunContext, ModelRetry

# Wrong — crashes the run, loses all prior work
@agent.tool
def lookup_record(ctx, record_id: str) -> dict:
    record = db.get(record_id)
    if not record:
        raise ValueError(f'Record {record_id} not found') # BAD

# Correct — LLM receives the error message and can self-correct
@agent.tool
def lookup_record(ctx: RunContext[None], record_id: str) -> dict:
    """Look up a record by ID.

    IDs are numeric strings, e.g. '1042'. The LLM reads this
    docstring — accurate guidance reduces ModelRetry frequency.
    """
    record = db.get(record_id)
    if not record:
        raise ModelRetry(
            f"No record found for ID '{record_id}'. "
            "Provide a numeric ID, e.g. '1042'."
        )
    return record

# retries= is the TOTAL budget across ALL tools and output validation.
# It is not per-tool. Three tools failing once each = 3 retries consumed.
agent = Agent('anthropic:claude-sonnet-4-6', output_type=str, retries=3)
```

⚠ Always catch `UnexpectedModelBehavior` at the call site — it is raised when the retry budget is exhausted. Inspect e.messages to see what the LLM was attempting, then decide whether to return a safe fallback or surface a 502.

2. RunContext — More Than Just deps

Most developers use `ctx.deps` and stop there. The other fields are often more useful — especially `ctx.retry` and `ctx.usage` for pipeline cost control:

```

from pydantic_ai import Agent, RunContext, ModelRetry

@agent.tool
def pipeline_tool(ctx: RunContext[AppDeps], query: str) -> str:
    """Example showing all RunContext fields relevant to pipeline
    work."""

    # ctx.deps - your injected dependencies (typed, IDE autocomplete
    works)
    user_id = ctx.deps.user_id
    rows = ctx.deps.db.execute(query).fetchall()

    # ctx.retry - which attempt is this? (0 = first attempt, 1 = first
    retry)
    # Use to log, adjust strategy, or fail fast after N retries.
    if ctx.retry > 0:
        print(f'Retry #{ctx.retry} - narrowing query scope')

    # ctx.usage - cumulative token count for the entire run so far.
    # Enforce a cost guard to prevent runaway pipelines.
    if (ctx.usage.input_tokens or 0) + (ctx.usage.output_tokens or 0) >
10_000:
        raise ModelRetry(
            'Token budget exceeded. Provide a shorter, more specific
query.'
        )

    # ctx.messages - full message history up to this point
    # Useful for logging or for tools that need conversation context.
    msg_count = len(ctx.messages)

    return f'Found {len(rows)} rows ({msg_count} messages,
{(ctx.usage.input_tokens or 0) + (ctx.usage.output_tokens or 0)} tokens)'

```

3. sync vs async — A Critical Rule

PydanticAI provides both `agent.run_sync()` (blocking) and `await agent.run()` (non-blocking). Mixing them is a common and painful mistake:

```

# WRONG - run_sync() inside an async def blocks the ENTIRE event loop.
# All other concurrent tasks, requests, or agents are frozen until
# the LLM responds. In a multi-agent pipeline, this serialises everything
# and can trigger timeouts on other concurrent operations.
async def handle_pipeline_stage(data: str):
    result = agent.run_sync(data)    # blocks event loop - BAD
    return result.output

# CORRECT - non-blocking; other coroutines continue while waiting
async def handle_pipeline_stage(data: str):
    result = await agent.run(data)   # yields control - GOOD
    return result.output

# run_sync() is only appropriate in scripts, CLI tools, and tests.
# In any async context (FastAPI, asyncio pipelines), always use await.

```

```
# To run multiple agents concurrently – asyncio.gather() is free:
# each await agent.run() yields control; Python runs the others.
results = await asyncio.gather(
    make_ingest_agent().run(prompt_a),
    make_ingest_agent().run(prompt_b),
    make_ingest_agent().run(prompt_c),
)
```

⚠️ *run_sync() inside async def is one of the most common PydanticAI mistakes in pipeline code. It does not raise an error — it silently degrades performance and can cause timeout cascades in concurrent workloads.*

4. ModelSettings — Timeouts and Token Limits

Always set explicit `ModelSettings` per agent. Different pipeline stages have very different requirements — a routing agent and a multi-step reasoning agent should never share the same settings.

```
from pydantic_ai.settings import ModelSettings

# Routing / classification – cheap, fast, deterministic
router = Agent(
    'anthropic:claude-haiku-4-5',          # cheapest model for routing
    output_type=RoutingDecision,
    model_settings=ModelSettings(
        temperature=0.0,    # deterministic – same input → same decision
        max_tokens=256,     # routing output is short
        timeout=10.0,       # fail fast if the API is slow
    ),
)

# Multi-step reasoning – expensive, slow, needs room to think
reason_agent = Agent(
    'anthropic:claude-opus-4-6',          # best model for complex
    reasoning
    output_type=AnalysisResult,
    model_settings=ModelSettings(
        temperature=0.0,    # deterministic for structured JSON output
        max_tokens=16000,   # must fit large JSON arrays – truncation =
broken
        timeout=120.0,      # complex reasoning legitimately takes 60-90s
    ),
)

# Override settings per-call when needed
result = reason_agent.run_sync(
    prompt,
    model_settings=ModelSettings(max_tokens=8000), # override for this
call
)
```

Task type	temperature	max_tokens	timeout
Routing / classification	0.0	128–256	5–10s

Task type	temperature	max_tokens	timeout
Structured extraction	0.0	512–4096	10–20s
Summarisation / Q&A	0.1–0.3	512–2048	20–30s
Multi-step reasoning, large JSON	0.0–0.1	8192–16000	60–120s
Creative / generative	0.7–1.0	2048+	30–60s

5. FallbackModel — Provider Resilience

For pipelines where uptime matters, `FallbackModel` chains providers automatically. If the primary provider returns a rate limit error, network error, or timeout, PydanticAI retries transparently with the next provider in the chain — no custom retry code required.

Note on model format: throughout this document, agents use the provider-prefixed string shorthand (e.g. 'anthropic:claude-sonnet-4-6'). Two exceptions use the explicit model class instead: `FallbackModel` (which requires model objects, not strings) and `make_reason_agent()` (where a custom httpx timeout is needed on the underlying client). Both forms are valid PydanticAI; the string shorthand is simpler when no custom client configuration is needed.

```
from pydantic_ai.models.fallback import FallbackModel
from pydantic_ai.models.openai import OpenAIModel
from pydantic_ai.models.anthropic import AnthropicModel

# FallbackModel requires explicit model objects, not string shorthands.
# Tries left to right; falls back on rate limits, network errors,
# timeouts.
# Does NOT fall back on schema validation failures.
fallback_model = FallbackModel(
    AnthropicModel('claude-sonnet-4-6'), # primary
    OpenAIModel('gpt-4o'),               # fallback 1
)

agent = Agent(fallback_model, output_type=AnalysisResult)

# Both API keys must be in the environment:
# ANTHROPIC_API_KEY and OPENAI_API_KEY
# PydanticAI selects the correct key per provider automatically.
```

✓ *FallbackModel is the production-grade answer to 529 overloaded errors during Anthropic peak hours. Keep the chain short (2–3 providers) — each additional fallback adds latency to the worst-case failure path.*

6. Prompt Caching — Cutting Repeat Costs

Anthropic caches static system prompts at 10% of the normal input token cost — a 90% saving on repeated context. For pipelines that run the same agents many times, this is material. The

rule is simple: system prompts must be static; dynamic data (run IDs, timestamps, per-record content) belongs exclusively in the user message.

```
# CORRECT — static system prompt qualifies for caching.
# The prompt is baked in at agent definition time and never changes.
INGEST_SYSTEM_PROMPT = """
You are an ingestion agent. Classify the input record type and decide
which data sources to query. Return a structured IngestResult.
"""

agent = Agent(
    'anthropic:claude-sonnet-4-6',
    output_type=IngestResult,
    system_prompt=INGEST_SYSTEM_PROMPT,    # static → eligible for caching
)

# WRONG — injecting dynamic data into the system prompt breaks caching.
# Every run sees a different system prompt → no cache hits ever.
def bad_system_prompt(run_id: str) -> str:
    return f'You are an ingestion agent for run {run_id}.'    # BAD

# CORRECT — dynamic data goes in the user message, not the system prompt.
prompt = f'Process this record. Run ID: {run_id}. Data:
{json.dumps(data)}'
result = await agent.run(prompt)    # system prompt is static → cache hit
```

 **Prompt caching is enforced architecturally by Claude Managed Agents (system prompts are baked at deploy time) but requires discipline in local runners. If your system prompt contains any f-string or runtime variable, you are paying full price on every call.**

7. TestModel — Testing Without API Calls

Use `TestModel` in unit tests to run PydanticAI agents at zero cost and in milliseconds. Combined with the factory pattern, every agent in the pipeline is fully testable without real API credentials or live network access.

 **Create the agent ONCE and apply `.override()` to that same instance. Calling `make_ingest_agent()` twice — once for the override and once for the run — creates two different objects. The override applies to the first; the second ignores it and makes a live API call.**

```
from pydantic_ai.models.test import TestModel

def test_prompt_contains_required_fields():
    """Verify the prompt is assembled correctly without an API call.

    CORRECT: one agent instance, override applied to that instance.
    """
    agent = make_ingest_agent()    # one instance
    deps = PipelineState(record_ids=['rec_42'], user_id='usr_1')
    with agent.override(model=TestModel()):    # override THAT instance
        result = agent.run_sync('Process record', deps=deps)
        prompt = result.all_messages_json()
```

```

        assert b'rec_42' in prompt

def test_output_schema_enforced():
    """Verify the output_type schema is validated on the agent output."""
    agent = make_ingest_agent()          # one instance
    mock = TestModel(

custom_result_text='{"summary":"ok","confidence":0.9,"items":[]}',
    )
    with agent.override(model=mock):      # override THAT instance
        result = agent.run_sync('Process record')
        assert isinstance(result.output, IngestResult)
        assert result.output.confidence == 0.9

# TestModel runs in milliseconds, costs nothing, and is deterministic.
# Use it for all unit tests. Reserve real LLM calls for infrequent
integration
# tests that run against a non-production API key.

```

8. HTTP Transport Retries — pydantic_ai.retries

PydanticAI v1 ships an official retry module that handles transient HTTP failures at the transport level — below the agent, retrying individual requests within a single `agent.run()` rather than restarting the entire workflow. This is strictly better than wrapping `agent.run()` in a retry loop when the failure is a genuine transient network issue.

```

# pip install 'pydantic-ai-slim[retries]' ← installs tenacity

import httpx
from tenacity import retry_if_exception_type, stop_after_attempt,
wait_exponential
from pydantic_ai import Agent
from pydantic_ai.models.anthropic import AnthropicModel
from pydantic_ai.providers.anthropic import AnthropicProvider
from pydantic_ai.retries import AsyncTenacityTransport, RetryConfig,
wait_retry_after

def create_resilient_anthropic_client() -> httpx.AsyncClient:
    """Build an httpx client with transport-level retry for Anthropic.

    Retries at the HTTP request level — individual streaming sessions
    within agent.run() — without restarting the whole agent workflow.
    wait_retry_after respects Retry-After headers from 429 responses;
    falls back to exponential backoff when no header is present.
    """
    transport = AsyncTenacityTransport(
        config=RetryConfig(
            retry=retry_if_exception_type((
                httpx.HTTPStatusError,    # 429, 502, 503, 504
                httpx.ConnectError,       # TCP connection refused
                httpx.ReadError,          # connection reset by peer
                httpx.TimeoutException,   # read/connect timeout
            )),
    ),

```

```

        wait=wait_retry_after(
            fallback_strategy=wait_exponential(multiplier=1, max=60),
            max_wait=300,
        ),
        stop=stop_after_attempt(5),
        reraise=True,
    ),
    validate_response=lambda r: r.raise_for_status() if r.status_code
in (429, 502, 503, 504) else None,
)
return httpx.AsyncClient(transport=transport)

# Wire the client into the AnthropicModel via AnthropicProvider
client = create_resilient_anthropic_client()
model = AnthropicModel(
    'claude-sonnet-4-6',
    provider=AnthropicProvider(http_client=client),
)
agent = Agent(model, output_type=AnalysisResult)

```

 **Critical distinction for pipeline developers: transport-level retries address genuine transient failures (brief network drops, momentary server overload). They do NOT fix pipeline-level rate limiting from too many simultaneous sessions. If `httpx.ReadError` is caused by the API dropping your connection because you have 40 sessions in-flight, retrying the dropped request adds session 41. The root fix is always architectural — reduce session count.**

The official PydanticAI docs explicitly list `httpx.ReadError` as a retryable network error (retries.md). For isolated agents handling occasional transient drops, this is correct. For high-throughput multi-agent pipelines, the same error may indicate volume-induced rate limiting — and the two failure modes produce identical error messages. Use Logfire to measure your session count under load before deciding whether retries or architecture is the right fix.

Putting It Together: A Complete `run_pipeline()` Function

All the patterns above connect at the pipeline level. This is the end-to-end orchestration function that ties every stage together. Reading it shows exactly why the session count is ~4 rather than 31–42.

Two helpers appear throughout this section and in the agent factory functions. They are defined first so the rest of the code is self-contained:

```
import json
```

```

# — System prompts (static – qualifies for prompt caching)
# Keep these constants. Dynamic content goes in the user message, not here.

INGEST_SYSTEM_PROMPT = """
You are an ingestion agent. Classify the input record type and decide which data sources to query. Return a structured IngestResult.
"""

REASON_SYSTEM_PROMPT = """
You are an analysis agent. You receive pre-collected data from multiple sources. Analyze patterns, correlate references, and validate findings. Return a structured AnalysisResult with confidence scores.
"""

# — Prompt builders
# These assemble the dynamic user message from the pipeline state.
# All variable content goes here – never in the system prompt.

def build_ingest_prompt(state: PipelineState) -> str:
    """Build the INGEST user message from raw input records."""
    return (
        f'Process these {len(state.record_ids)} records: '
        f'{json.dumps(state.record_ids)}\n'
        f'User: {state.user_id}'
    )

def build_reason_prompt(state: PipelineState) -> str:
    """Bundle all pre-fetched data into the REASON agent's user message.

    This single call replaces what used to be 10-20 individual tool calls.
    All data collection already happened in run_prefetch() at zero LLM cost.
    """
    return (
        f'Collection results:\n{json.dumps(state.collect_results)}\n\n'
        f'Enrichment results:\n{json.dumps(state.enrich_results)}\n\n'
        f'Knowledge base data:\n{json.dumps(state.kb_data)}\n\n'
        f'Reference advisories:\n{json.dumps(state.ref_data)}\n\n'
        f'Status data:\n{json.dumps(state.status_data)}\n\n'
        'Perform pattern analysis, correlation, and validation. '
        'Return structured JSON matching the AnalysisResult schema.'
    )

import asyncio, json
from dataclasses import dataclass, field
from typing import Any
from pydantic_ai.exceptions import UnexpectedModelBehavior
from pydantic import ValidationError

@dataclass
class PipelineState:
    """Typed shared state passed between pipeline stages.

```

```

Each stage reads from and writes to this object.
It is the single source of truth for one pipeline run.
"""
record_ids: list[str]
user_id: str
ingest_results: Any = field(default=None)
collect_results: Any = field(default=None)
enrich_results: Any = field(default=None)
kb_data: Any = field(default=None)
ref_data: Any = field(default=None)
status_data: Any = field(default=None)
reason_results: Any = field(default=None)

async def run_pipeline(state: PipelineState) -> PipelineState:
    """Execute the full pipeline for one batch of input records.

    Total API sessions: ~4 (INGEST ~3 + REASON 1).
    All other stages are direct Python – zero LLM sessions.
    """
    # Stage 1: INGEST – LLM classifies inputs, decides query strategy
    state = await run_ingest(state) # ~3 API sessions

    # Stage 2 & 3: COLLECT + ENRICH – pure Python, no LLM
    state = await run_collect(state) # 0 API sessions
    state = await run_enrich(state) # 0 API sessions

    # Stage 4: PRE-FETCH – gather all reasoning data in Python
    state = await run_prefetch(state) # 0 API sessions

    # Stage 5: REASON – one LLM call, no tools, all data pre-loaded
    state = await run_reason(state) # 1 API session

    # Stage 6 & 7: REPORT + TRANSFORM – pure Python, no LLM
    state = await run_report(state) # 0 API sessions
    state = await run_transform(state) # 0 API sessions

    return state

async def run_ingest(state: PipelineState) -> PipelineState:
    agent = make_ingest_agent() # fresh instance – never reuse
    try:
        result = await agent.run(build_ingest_prompt(state), deps=state)
        state.ingest_results = result.output
    except UnexpectedModelBehavior as e:
        raise RuntimeError(f'INGEST failed after retries: {e}') from e
    return state

async def run_prefetch(state: PipelineState) -> PipelineState:
    """Pre-fetch ALL data REASON needs – zero LLM API sessions.

    asyncio.gather() runs all subprocess calls concurrently.
    This is safe to parallelise because none of these touch the LLM API.
    """
    state.kb_data, state.ref_data, state.status_data = await
    asyncio.gather(
        run_tool('knowledge_base.py', 'lookup', *state.record_ids),

```

```
        run_tool('reference_lookup.py', 'query', *state.record_ids),
        run_tool('status_check.py', 'get', *state.record_ids),
    )
    return state

async def run_reason(state: PipelineState) -> PipelineState:
    """Run REASON: exactly one LLM call with all data, no tools."""
    agent = make_reason_agent()      # fresh instance, no tools registered
    prompt = build_reason_prompt(state)  # bundles all pre-fetched data
    try:
        result = await agent.run(prompt)
        state.reason_results = result.output
    except UnexpectedModelBehavior as e:
        # Retry budget exhausted – inspect e.messages to see what LLM
        returned
        raise RuntimeError(f'REASON failed after retries: {e}') from e
    return state

# Entry point
async def main():
    state = PipelineState(record_ids=['rec_1', 'rec_2'],
user_id='usr_42')
    result = await run_pipeline(state)
    print(f'Pipeline complete: {len(result.reason_results.items)}
results')
```

Summary: The Rules

Rule	Why
Count sessions first, not tokens	LLM API rate limits surface as TCP RSTs, not HTTP 429. Infrastructure debugging is a false trail.
Sessions, not tokens, kill pipelines	Tokens-per-minute is the limit developers are told about. Sessions-per-second is the limit that actually kills pipelines.
Each tool call = 1 more API session	N tool calls = N+1 API sessions per agent.run(). Account for this when designing.

Rule	Why
Retries compound the problem	Retrying a rate-limited session fires more sessions. The correct response is to reduce session count, not increase attempts.
Transport retries ≠ architecture fix	pydantic_ai.retries handles genuine transient drops (network blips). If ReadError is volume-induced rate limiting, transport retries add more sessions. Measure first with Logfire.
Not every stage needs an LLM	If removing the LLM produces identical output, it should never have been there. Route in Python; reason in LLM.
Pre-fetch data; reason once	Gather everything via Python subprocesses. Pass it all to the LLM in one prompt. One session.
Collapse adjacent reasoning stages	Three agents doing related analysis = one agent with a richer system prompt. Same reasoning, ~90% fewer API sessions.
Prompt efficiency has a ceiling	Efficiency prompts reduce waste within an agent. Only architecture reduces pipeline-level session count.
Use factory functions, not singletons	PydanticAI agents accumulate streaming state. Create a fresh instance per invocation.
Set explicit timeouts and max_retries=0	Prevent indefinite hangs and silent SDK-level duplicate requests.
Keep system prompts static for prompt caching	Anthropic caches static system prompts at 10% of normal input cost. Dynamic content belongs in the user message, not the system prompt.
Don't run LLM stages in parallel early on	Parallel agents multiply simultaneous sessions. Wait until you have measured your rate limit headroom.

Pre-Build Checklist

Before writing your first agent in a new multi-agent pipeline:

Architecture

- ☐ For each planned stage: does it require LLM reasoning, or is it data collection / routing / formatting?
- ☐ For each tool-using agent: can the data be pre-fetched in Python and passed in the prompt instead?
- ☐ Can adjacent reasoning stages that operate on the same data be merged into one agent?
- ☐ What is the worst-case total API session count per pipeline run? Write it down before you start coding.
- ☐ If you plan parallel agent stages, do the simultaneous session counts stay within your rate limit headroom?

Implementation

- ☐ All agents created via factory functions — never module-level singletons
- ☐ `AsyncAnthropic` / SDK clients have explicit `httpx.Timeout` and `max_retries=0`
- ☐ `max_tokens` is set high enough that structured JSON outputs will never truncate mid-array
- ☐ Retry logic distinguishes between transient errors (retry) and rate-limit resets (back off significantly or fail fast)
- ☐ There is logging or instrumentation that counts actual API sessions per run

Validation

- ☐ Run against a single target and check your API dashboard for actual session count
- ☐ Confirm the count matches your architectural estimate
- ☐ Only then scale to multiple targets or parallel stages

Closing Thoughts

The architectural fix, once we understood the problem, took a few hours to implement. The debugging that preceded it cost days.

The pipeline that emerged is leaner, faster, cheaper, and — because each stage has a single clear responsibility — easier to test and reason about than the original design. The connection errors haven't returned. Every run since the redesign has completed cleanly.

The PydanticAI + Anthropic API combination is genuinely powerful for multi-agent pipelines. But the API's rate limits are not the same shape as the rate limits you encounter with REST services. When you cross the streaming session threshold, you don't get a 429 you can inspect — you get a TCP reset that looks like network instability. Knowing this in advance changes how you design.

 *The most valuable exercise: before writing code, draw your pipeline's session budget. Count exactly how many API streaming sessions each run will consume. If the number surprises you, the architecture needs rethinking — before the pipeline does.*

Appendix A — PydanticAI Practical Reference Guide

This appendix distills the PydanticAI framework into a self-contained reference. It is structured as a progressive guide — from the import layer map through dependency injection, validation, error handling, FastAPI integration, observability, and a framework comparison. Sections A.1 and A.6 recap content discussed in the main body; all other sections are additive.

A.1 The Three-Layer Model

The single biggest source of confusion when starting with PydanticAI is conflating two separate packages that share the Pydantic name.

Layer	Package	Responsibility	Import from
Schema	pydantic	Define data shapes, validate data, raise <code>ValidationError</code>	from pydantic import <code>BaseModel</code> , <code>Field</code>
Agent	pydantic_ai	Wrap LLMs, run tools, enforce output schemas, handle retries	from pydantic_ai import <code>Agent</code> , <code>RunContext</code>
Observability	logfire	OTel tracing for every agent run, tool call, LLM request	import logfire

⚠️ *pydantic ≠ pydantic_ai. Never write from pydantic_ai import BaseModel or from pydantic import Agent — both fail at runtime. pip install pydantic-ai installs both packages but they have separate import namespaces.*

```

from pydantic import BaseModel, Field, field_validator  # schema layer
from pydantic_ai import Agent, RunContext, ModelRetry  # agent layer
from pydantic_ai.exceptions import UnexpectedModelBehavior
from pydantic_ai.models.anthropic import AnthropicModel
from pydantic_ai.settings import ModelSettings
import logfire  # observability layer

# WRONG — these do NOT exist:
# from pydantic_ai import BaseModel      ← use pydantic
# from pydantic import Agent             ← use pydantic_ai
# from pydantic_ai import ValidationError ← it lives in pydantic

```

Environment setup: load your API key before constructing any Agent. If you define agents at module level, this must be the very first import in the file. Without it, `Agent()` silently succeeds but `.run()` fails with a 401 that looks identical to a bad API key.

```

pip install pydantic-ai python-dotenv

from dotenv import load_dotenv
load_dotenv()  # must run before Agent() is constructed

agent = Agent('anthropic:claude-sonnet-4-6', output_type=str)

```

A.2 Building Agents

A.2.1 Minimal Agent — Structured Output

The simplest useful agent returns a typed Pydantic model instead of raw text. The framework enforces the schema and retries automatically on validation failure.

```
from pydantic import BaseModel
from pydantic_ai import Agent

class ReviewResult(BaseModel):
    title: str
    year: int
    rating: float          # validated: must be a float
    summary: str
    recommended: bool

agent = Agent(
    'anthropic:claude-sonnet-4-6',
    output_type=ReviewResult,
    system_prompt='You are a critic. Always respond in structured form.',
)

result = agent.run_sync('Review the latest model release')
review = result.output  # type: ReviewResult — fully typed
print(review.rating, review.recommended)
# IDE provides autocomplete; mypy catches type errors at write time
```

A.2.2 Agent with Tools — @tool vs @tool_plain

Tools are plain Python functions. PydanticAI validates arguments before calling them and validates return values before passing results back to the LLM. The docstring is read by the LLM as guidance — keep it accurate.

Decorator	When to use	Has RunContext?
@agent.tool_plain	Pure function — no framework state needed	No
@agent.tool	Needs ctx.deps, ctx.retry, ctx.usage, ctx.messages	Yes

```
from pydantic_ai import Agent, RunContext
import httpx

agent = Agent('anthropic:claude-sonnet-4-6', output_type=str)

@agent.tool_plain          # pure function — no RunContext needed
def fetch_page(url: str) -> str:
    """Fetch the text content of a URL. Returns first 4000 characters."""
    resp = httpx.get(url, timeout=10)
    return resp.text[:4000]

@agent.tool                # has access to RunContext
def current_date(ctx: RunContext[None]) -> str:
    """Return today's date in ISO format."""
    from datetime import date
```

```

    return date.today().isoformat()

result = agent.run_sync('What is the latest PydanticAI release date?')

```

A.2.3 Dependency Injection — Multi-tenant Pattern

Dependencies let you pass database connections, API clients, or user context into tools without globals. The agent is generic over its deps type — the same agent works across users or tenants with different injected values. This is also how you make agents fully testable: inject a mock database and no live infrastructure is needed.

```

from dataclasses import dataclass
from pydantic import BaseModel
from pydantic_ai import Agent, RunContext

@dataclass
class UserContext:
    user_id: str
    tenant_id: str
    db_conn: object
    allowed_tables: list[str]

class QueryResult(BaseModel):
    sql: str
    explanation: str
    row_count: int

# Agent is generic over its deps type
agent: Agent[UserContext, QueryResult] = Agent(
    'anthropic:claude-sonnet-4-6',
    output_type=QueryResult,
    system_prompt='Generate safe read-only SQL for the user tenant.',
)

@agent.system_prompt          # dynamic system prompt reads deps at
runtime
def add_schema_context(ctx: RunContext[UserContext]) -> str:
    tables = ', '.join(ctx.deps.allowed_tables)
    return f'Tenant {ctx.deps.tenant_id}. Available tables: {tables}.'

@agent.tool
def run_query(ctx: RunContext[UserContext], sql: str) -> dict:
    """Execute the SQL and return rows. Only runs on allowed_tables."""
    rows = ctx.deps.db_conn.execute(sql).fetchall()
    return {'rows': rows, 'count': len(rows)}

# Inject deps at call time — swap in tests without changing agent code
ctx = UserContext(
    user_id='u_123', tenant_id='acme',
    db_conn=get_db(),          # your DB connection factory
    allowed_tables=['orders', 'products'],
)
result = agent.run_sync('Show last month top 5 orders', deps=ctx)

```

A.2.4 Multi-turn Conversation

Pass the accumulated message history back on each call to maintain context across turns. The history grows with each exchange; trim old turns or persist to a database for long sessions.

```
from pydantic_ai import Agent
from pydantic_ai.messages import ModelMessage

agent = Agent('anthropic:claude-sonnet-4-6', output_type=str)
history: list[ModelMessage] = []

def chat(user_input: str) -> str:
    result = agent.run_sync(user_input, message_history=history)
    history.extend(result.new_messages()) # accumulate for next turn
    return result.output

print(chat('My name is Priya and I work in Singapore.'))
print(chat('What time zone for a 9 AM London standup?'))
print(chat('Remind me what city I mentioned.')) # agent remembers

# For long sessions: persist with model_dump() and reload with
model_validate()
# history_json = [m.model_dump() for m in history]
```

A.2.5 Agent Composition — Router Pattern

Do not give one agent 20 tools. Compose focused specialist agents behind a cheap router. The router classifies intent using a small fast model; specialist agents handle each case with the right model and system prompt.

```
from pydantic_ai import Agent
from pydantic import BaseModel
from typing import Literal

# Specialist agents
class CodeReview(BaseModel):
    issues: list[str]
    severity: Literal['low', 'medium', 'high']
    fixed_code: str | None

class DocSummary(BaseModel):
    tldr: str
    key_points: list[str]

code_agent = Agent('anthropic:claude-sonnet-4-6', output_type=CodeReview,
    system_prompt='You are a senior engineer doing code review.')
doc_agent = Agent('anthropic:claude-sonnet-4-6', output_type=DocSummary,
    system_prompt='You are a technical writer. Summarise clearly.')

# Router — cheap fast model for classification only
class Intent(BaseModel):
    category: Literal['code_review', 'summarise', 'other']
    content: str

router = Agent('anthropic:claude-haiku-4-5', output_type=Intent,
```

```

    system_prompt='Classify the request and extract relevant content.')

async def handle_request(user_message: str):
    intent = (await router.run(user_message)).data
    match intent.category:
        case 'code_review': return (await
code_agent.run(intent.content)).data
        case 'summarise':    return (await
doc_agent.run(intent.content)).data
        case _:              return None

```

A.3 Input & Output Validation

A.3.1 Validate Input Before the LLM Sees It

Validate prompt parameters with a Pydantic model at the call site. Fail fast on bad input before burning any tokens.

```

from pydantic import BaseModel, Field, field_validator
from pydantic_ai import Agent
from typing import Literal

class AnalysisRequest(BaseModel):
    text: str = Field(min_length=10, max_length=10_000)
    language: Literal['en', 'fr', 'de', 'ja'] = 'en'
    max_bullets: int = Field(default=5, ge=1, le=20)

    @field_validator('text')
    @classmethod
    def no_pii_patterns(cls, v: str) -> str:
        import re
        if re.search(r'[0-9]{16}', v):    # crude card number check
            raise ValueError('Input appears to contain a card number')
        return v.strip()

class AnalysisResult(BaseModel):
    summary: str
    bullets: list[str]
    sentiment: Literal['positive', 'neutral', 'negative']
    word_count: int

agent = Agent('anthropic:claude-sonnet-4-6', output_type=AnalysisResult)

def analyse(req: AnalysisRequest) -> AnalysisResult:
    # ValidationError raised here before any API call is made
    prompt = (
        f'Analyse the following {req.language} text. '
        f'Provide up to {req.max_bullets} bullet points.\n\n{req.text}'
    )
    return agent.run_sync(prompt).data

```

A.3.2 Nested Models, Enums, and Cross-field Validators

Use `model_validator` for cross-field business rules that cannot be expressed as per-field constraints. These run after all fields are parsed but before the result is returned to your code.

```
from pydantic import BaseModel, Field, model_validator
from pydantic_ai import Agent
from enum import Enum

class Priority(str, Enum):
    LOW = 'low'; MEDIUM = 'medium'; HIGH = 'high'; CRITICAL = 'critical'

class ActionItem(BaseModel):
    title: str = Field(max_length=80)
    owner: str | None = None
    due_days: int = Field(ge=0, le=365)
    priority: Priority

class MeetingMinutes(BaseModel):
    title: str
    attendees: list[str] = Field(min_length=1)
    decisions: list[str]
    action_items: list[ActionItem]

    @model_validator(mode='after')
    def critical_items_must_have_owner(self) -> 'MeetingMinutes':
        for item in self.action_items:
            if item.priority == Priority.CRITICAL and not item.owner:
                raise ValueError(
                    f"Critical action '{item.title}' must have an owner"
                )
        return self

agent = Agent('anthropic:claude-sonnet-4-6', output_type=MeetingMinutes)
```

Validator type	When it runs	Has RunContext?	Best for
field_validator	On each field after parsing	No	Format checks, sanitisation, PII blocking
model_validator	After all fields parsed	No	Cross-field business rules
result_validator (PydanticAI)	After model_validator, before return	Yes	DB lookups, live API checks, real-data substitution

A.3.3 Union Output Types — Typed Success or Error

When the LLM may legitimately be unable to complete a task, use a Union return type to force explicit typed handling of both outcomes at the call site.

```
from pydantic import BaseModel, Field
from pydantic_ai import Agent
from typing import Union, Literal
```

```

class TranslationSuccess(BaseModel):
    status: Literal['success'] = 'success'
    translated_text: str
    detected_language: str
    confidence: float = Field(ge=0.0, le=1.0)

class TranslationError(BaseModel):
    status: Literal['error'] = 'error'
    reason: str
    suggestion: str | None = None

agent = Agent(
    'anthropic:claude-sonnet-4-6',
    output_type=Union[TranslationSuccess, TranslationError],
    system_prompt='Translate to English. If impossible, return an
error.',
)

res = agent.run_sync('Bonjour le monde').data
match res:
    case TranslationSuccess(translated_text=t, confidence=c):
        print(f'Translated ({c:.0%}): {t}')
    case TranslationError(reason=r):
        print(f'Failed: {r}')

```

A.4 Errors & Retries

A.4.1 ModelRetry — Recoverable Tool Errors

Raise `ModelRetry` instead of a plain exception when a tool cannot satisfy the request. The framework feeds your message back to the LLM which can correct course. A plain `ValueError` aborts the run entirely.

```

from pydantic_ai import Agent, RunContext, ModelRetry

agent = Agent('anthropic:claude-sonnet-4-6', output_type=str, retries=3)

@agent.tool
def lookup_record(ctx: RunContext[None], record_id: str) -> dict:
    """Fetch a record by ID. IDs are numeric strings, e.g. '1042'."""
    record = db.get(record_id)
    if not record:
        raise ModelRetry(
            f"No record for ID '{record_id}'. "
            "Try a numeric ID such as '1042'.",
        )
    return record

# retries= is the TOTAL budget across ALL tools + output validation.
# It is NOT per-tool - 3 tools failing once each = 3 retries consumed.

```

A.4.2 Catching Exhausted Retries — Exception Reference

```

from pydantic_ai.exceptions import UnexpectedModelBehavior
from pydantic import ValidationError

def safe_run(text: str):
    try:
        return agent.run_sync(text).data
    except UnexpectedModelBehavior as e:
        # LLM kept returning invalid output after retries exhausted.
        # e.messages contains the full conversation – inspect it.
        print(f'Model misbehaved: {e}')
        print(f'Last messages: {e.messages}')
        return None
    except ValidationError as e:
        # Your INPUT model failed – programmer error, always re-raise.
        raise
    except Exception as e:
        # Network, rate limit, auth error, etc.
        print(f'Unexpected: {type(e).__name__}: {e}')
        return None

```

Exception	Cause	Correct response
UnexpectedModelBehavior	LLM failed validation after retry budget exhausted	Log e.messages, return safe fallback or raise 502
ValidationError (pydantic)	Your INPUT model is wrong — programmer error	Always re-raise. Fix the calling code.
httpx / network errors	Connectivity, rate limit, API key invalid	Log, return fallback, add exponential backoff

A.4.3 result_validators — Post-parse Business Logic

`result_validators` run after Pydantic validates the schema but before the result is returned. Unlike `field_validators`, they have full access to `RunContext` — so you can hit a database or call an API to verify the LLM's output.

```

from pydantic_ai import Agent, RunContext, ModelRetry
from pydantic import BaseModel
from dataclasses import dataclass

class ProductRecommendation(BaseModel):
    product_id: str
    reason: str
    price_estimate: float

@dataclass
class CatalogueDeps:
    catalogue_db: object
    user_budget: float

agent: Agent[CatalogueDeps, ProductRecommendation] = Agent(

```

```

        'anthropic:claude-sonnet-4-6',
        output_type=ProductRecommendation,
        retries=3,
    )

    @agent.result_validator # runs AFTER Pydantic schema validation
    async def validate_product_exists(
        ctx: RunContext[CatalogueDeps],
        result: ProductRecommendation,
    ) -> ProductRecommendation:
        product = await ctx.deps.catalogue_db.get(result.product_id)
        if not product:
            raise ModelRetry(
                f"Product '{result.product_id}' does not exist. "
                'Only recommend products from the catalogue.',
            )
        if result.price_estimate > ctx.deps.user_budget:
            raise ModelRetry('Price exceeds user budget. Suggest a cheaper option.')
        result.price_estimate = product.actual_price # substitute real data
        return result

```

A.5 RunContext — The Full Picture

Most developers use `ctx.deps` and stop there. The other fields are often more useful — especially `ctx.retry` and `ctx.usage` for pipeline cost control. Think of `RunContext` as the request object in FastAPI: it carries everything about the current run without manual threading.

```

from pydantic_ai import Agent, RunContext, ModelRetry

@agent.tool
def pipeline_tool(ctx: RunContext[AppDeps], query: str) -> str:

    # ctx.deps — your injected dependencies (typed, IDE autocomplete works)
    user_id = ctx.deps.user_id
    rows = ctx.deps.db.execute(query).fetchall()

    # ctx.retry — which attempt is this? (0 = first, 1 = first retry)
    if ctx.retry > 0:
        print(f'Retry #{ctx.retry} — narrowing scope for user {user_id}')

    # ctx.usage — RunUsage with live request-level counts.
    # Fields: .input_tokens, .output_tokens, .requests
    # Use to enforce cost guards. Counts are cumulative for the run so far.
    if (ctx.usage.input_tokens or 0) + (ctx.usage.output_tokens or 0) > 10_000:
        raise ModelRetry(
            'Token budget exceeded. Provide a shorter, more specific query.'
        )

    # ctx.messages — full conversation history up to this point.
    # Useful for logging or for tools that need prior context.

```

```

msg_count = len(ctx.messages)

# ctx.model — the model instance (useful in fallback chains).
model_name = str(ctx.model)

return f'{rows} ({msg_count} msgs, {(ctx.usage.input_tokens or 0) +
(ctx.usage.output_tokens or 0)} tokens used)'

```

A.6 Async, Settings, and Testing

A.6.1 sync vs async

⚠️ Never call `run_sync()` inside an `async def` — it blocks the entire event loop, freezing all other concurrent tasks. In `async` contexts (FastAPI, `asyncio` pipelines) always use `await agent.run()`.

```

# WRONG — blocks event loop
async def handle(text: str):
    result = agent.run_sync(text)    # BAD

# CORRECT — non-blocking
async def handle(text: str):
    result = await agent.run(text)   # GOOD

# Concurrent agents — asyncio.gather() is free (pure I/O yield)
results = await asyncio.gather(
    agent.run(prompt_a),
    agent.run(prompt_b),
    agent.run(prompt_c),
)

```

A.6.2 ModelSettings Reference

Task type	temperature	max_tokens	timeout
Routing / intent classification	0.0	128–256	5–10s
Structured extraction	0.0	512–4096	10–20s
Summarisation / Q&A	0.1–0.3	512–2048	20–30s
Multi-step reasoning, large JSON	0.0–0.1	8192–16000	60–120s
Creative / generative	0.7–1.0	2048+	30–60s

A.6.3 TestModel — Zero API Cost Testing

⚠ Create the agent once and apply `.override()` to that same instance. Calling `make_agent()` twice inside a with block creates two different objects — the override on the first does not apply to the second.

```
from pydantic_ai.models.test import TestModel

def test_output_schema():
    agent = make_ingest_agent()          # one instance
    with agent.override(model=TestModel(

custom_result_text='{"summary":"ok","confidence":0.9,"items":[]}',
    )):
        result = agent.run_sync('Process record')
        assert isinstance(result.output, IngestResult)
        assert result.output.confidence == 0.9

# TestModel: milliseconds, zero cost, deterministic.
# Use for all unit tests; reserve real LLM calls for infrequent
integration tests.
```

A.7 FastAPI Integration — Capstone Pattern

FastAPI validates HTTP inputs with Pydantic. PydanticAI validates LLM outputs with Pydantic. Both use the same library, so the schema language is identical throughout — from HTTP request body through LLM response to database write.

```
from contextlib import asynccontextmanager
from dataclasses import dataclass
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from pydantic_ai import Agent, RunContext, ModelRetry
from pydantic_ai.exceptions import UnexpectedModelBehavior
from pydantic_ai.settings import ModelSettings
from dotenv import load_dotenv
load_dotenv()  # must precede Agent()

# 1. Schemas
class TicketRequest(BaseModel):          # FastAPI validates from HTTP body
    description: str = Field(min_length=10, max_length=2000)
    user_id: str
    project_id: str

class Ticket(BaseModel):                 # PydanticAI validates from LLM
    title: str = Field(max_length=80)
    category: str
    severity: int = Field(ge=1, le=5)
    suggested_assignee: str | None = None

# 2. Deps
@dataclass
class AgentDeps:
    project_id: str
```

```

    user_id: str
    team_members: list[str]

# 3. Agent
agent: Agent[AgentDeps, Ticket] = Agent(
    'anthropic:claude-sonnet-4-6',
    output_type=Ticket,
    retries=2,
    model_settings=ModelSettings(temperature=0.0, timeout=20.0),
)

@agent.system_prompt
def build_prompt(ctx: RunContext[AgentDeps]) -> str:
    members = ', '.join(ctx.deps.team_members)
    return (
        f'Triage tickets for project {ctx.deps.project_id}. '
        f'Team: {members}. Severity 1=low, 5=critical.'
    )

@agent.result_validator
async def validate_assignee(ctx: RunContext[AgentDeps], result: Ticket)
-> Ticket:
    if result.suggested_assignee and \
        result.suggested_assignee not in ctx.deps.team_members:
        raise ModelRetry(
            f"'{result.suggested_assignee}' is not on the team. "
            f"Choose from: {ctx.deps.team_members}"
        )
    return result

# 4. FastAPI route
@asynccontextmanager
async def lifespan(app: FastAPI):
    yield # initialise DB pools and caches here

app = FastAPI(lifespan=lifespan)

@app.post('/tickets', response_model=Ticket)
async def create_ticket(req: TicketRequest) -> Ticket:
    team = await get_team(req.project_id) # your DB lookup - returns
    list[str]
    deps = AgentDeps(
        project_id=req.project_id,
        user_id=req.user_id,
        team_members=team,
    )
    try:
        result = await agent.run(req.description, deps=deps)
        return result.output
    except UnexpectedModelBehavior as e:
        raise HTTPException(502, detail=f'LLM failed: {e}')
    except Exception as e:
        raise HTTPException(500, detail=f'Error: {type(e).__name__}')

```

HTTP status	Cause	Source
422 Unprocessable Entity	TicketRequest failed Pydantic validation	FastAPI (automatic)
502 Bad Gateway	LLM failed after all retries	UnexpectedModelBehavior
500 Internal Server Error	Network, auth, or unexpected bug	Generic except
200 OK	Ticket returned and validated	result.output

A.7.1 Background Tasks — Async Pipeline Serving

A pipeline that runs 30–90 seconds must never block an HTTP request. Clients time out; load balancers terminate long-lived connections. The correct pattern is to accept the request immediately with a 202 Accepted and run the pipeline in a FastAPI BackgroundTask. The client polls a status endpoint for the result.

This pattern also isolates pipeline failures from HTTP layer failures. A connection reset inside the pipeline does not surface as a 5xx to the client — it surfaces as a failed task in your polling store, where it can be retried or escalated independently.

```

from fastapi import FastAPI, BackgroundTasks, HTTPException
from pydantic import BaseModel
from uuid import uuid4
from typing import Any

# In production: replace with Redis or a database. In-memory is
# lost on process restart and does not work across multiple workers.
task_store: dict[str, dict[str, Any]] = {}

class PipelineRequest(BaseModel):
    record_ids: list[str]
    user_id: str

class TaskStatus(BaseModel):
    task_id: str
    status: str          # 'pending' | 'running' | 'complete' |
    'failed'
    result: Any = None
    error: str | None = None

@app.post('/pipeline', status_code=202, response_model=TaskStatus)
async def start_pipeline(req: PipelineRequest, tasks: BackgroundTasks):
    """Accept immediately. Run pipeline in background. Client polls for
    result."""
    task_id = str(uuid4())
    task_store[task_id] = {'status': 'pending'}
    tasks.add_task(_run_pipeline_task, task_id, req)
    return TaskStatus(task_id=task_id, status='pending')

@app.get('/pipeline/{task_id}', response_model=TaskStatus)
async def poll_pipeline(task_id: str):
    task = task_store.get(task_id)

```

```

    if not task:
        raise HTTPException(404, detail=f'Unknown task: {task_id}')
    return TaskStatus(task_id=task_id, **task)

async def _run_pipeline_task(task_id: str, req: PipelineRequest) -> None:
    task_store[task_id] = {'status': 'running'}
    try:
        state = PipelineState(record_ids=req.record_ids,
                               user_id=req.user_id)
        result = await run_pipeline(state)
        task_store[task_id] = {'status': 'complete', 'result': result}
    except Exception as exc:
        task_store[task_id] = {'status': 'failed', 'error': str(exc)}

```

 **Rate limiting at the FastAPI layer prevents Anthropic connection saturation before it starts. *slowapi* (`pip install slowapi`) adds per-user request rate limiting in two lines: `app.state.limiter = Limiter(key_func=get_remote_address) + @limiter.limit("5/minute")` on the route. Throttle inbound requests to a rate your session budget can absorb.**

A.8 Observability with Logfire

Two lines instrument every agent run, tool call, and LLM request with full OpenTelemetry spans. Logfire works with any OTEL-compatible backend: Jaeger, Datadog, Honeycomb, Grafana Tempo.

```

import logfire
from pydantic_ai import Agent

logfire.configure() # reads LOGFIRE_TOKEN from env
logfire.instrument_pydantic_ai() # zero code changes to existing agents

# What you see in Logfire for one pipeline run:
# run_pipeline
# └─ pydantic_ai.run (INGEST)           ← ~3 streaming sessions
#   └─ pydantic_ai.llm_request         ← session 1: decide tool call
#     └─ pydantic_ai.tool_call         ← source_query executed
#       └─ pydantic_ai.llm_request     ← session 2: decide tool call
#         └─ pydantic_ai.tool_call     ← classify_record executed
#           └─ pydantic_ai.llm_request ← session 3: final answer
# └─ pydantic_ai.run (REASON)          ← 1 streaming session
#   └─ pydantic_ai.llm_request         ← session 1: no tool calls

# Total sessions: 4. If you see more, a tool call is happening
# somewhere you didn't expect — Logfire makes it immediately visible.

# Custom span wrapping a pipeline stage:
with logfire.span('run_pipeline', batch_id='batch_42'):
    result = await run_pipeline(state)

```

A.8 Observability with Logfire — Deep Dive

The basic setup above gives you the agent span tree. Three additional instrumentation steps give you the diagnostic power needed to investigate connection resets, monitor session budgets, and distinguish rate-limiting patterns from genuine network transients.

A.8.2 HTTP-level visibility — see raw Anthropic requests and connection resets

Instrument the httpx transport to capture every HTTP request made to the Anthropic API. `capture_all=True` records status codes and headers — letting you distinguish a genuine 429 from a TCP RST that drops the connection with no HTTP status code at all.

```
import logfire
from anthropic import AsyncAnthropic

logfire.configure()
logfire.instrument_pydantic_ai()

# Instrument ALL httpx clients — includes those used internally by
# PydanticAI and the Anthropic SDK. No code changes to existing agents.
logfire.instrument_httpx(capture_all=True)

# Also instrument any raw AsyncAnthropic client used outside PydanticAI:
raw_client = AsyncAnthropic()
logfire.instrument_anthropic(raw_client)

# Filter for resets in Logfire live view:
# WHERE message LIKE '%ReadError%' OR message LIKE '%connection reset%'

# Clustered resets (same minute) → volume rate-limiting → reduce sessions
# Sparse random resets           → genuine transient → transport retry
OK
```

A.8.3 Session budget metric — alert before the architecture regresses

Register a histogram metric at startup to record sessions consumed per pipeline run. If sessions creep back toward 40 after a future code change, the alert fires before it causes failures in production.

```
import logfire

# Register once at module level. Reuse across all pipeline invocations.
sessions_metric = logfire.metric_histogram(
    'pipeline.sessions_per_run',
    unit='sessions',
    description='Anthropic API streaming sessions per pipeline run',
)
reset_counter = logfire.metric_counter(
    'pipeline.connection_resets',
    description='ReadError connection resets observed per run',
)

async def run_pipeline_instrumented(state: PipelineState) -> PipelineState:
    with logfire.span('run_pipeline', user_id=state.user_id):
        result = await run_pipeline(state)
        # Count llm_request spans in result to get actual session total
```

```

    sessions_metric.record(session_count, {'user': state.user_id})
    return result

# Logfire dashboard actions:
# 1. Histogram chart on 'pipeline.sessions_per_run'
# 2. Alert: p95 > 6 → notify Slack (session creep detected)
# 3. Overlay 'pipeline.connection_resets' — spikes should correlate

```

A.8.4 SQL diagnostic — confirm the failure pattern

Logfire stores all spans as queryable SQL records. These two queries confirm definitively whether connection resets are volume-induced or genuinely random.

```

-- 1. Resets per minute — do they cluster at busy periods?
SELECT date_trunc('minute', start_timestamp) AS minute,
       count(*) AS resets
FROM records
WHERE message LIKE '%ReadError%'
       OR message LIKE '%connection reset%'
GROUP BY 1 ORDER BY 1;
-- Tight clusters → rate-limiting. Sparse → genuine transient.

-- 2. Session count per run — is it creeping back up?
SELECT attributes->>'user_id' AS user,
       count(*) AS llm_requests
FROM records
WHERE otel_scope_name = 'pydantic_ai'
       AND span_name = 'pydantic_ai.llm_request'
GROUP BY trace_id, attributes->>'user_id'
ORDER BY llm_requests DESC LIMIT 20;
-- Any run > 5 has an unexpected tool call. Find it in the trace view.

```

A.9 Framework Comparison — PydanticAI vs LangChain vs LlamaIndex

PydanticAI does not exist in isolation. Understanding where it fits relative to LangChain and LlamaIndex clarifies when to use it and when to combine it with others.

Dimension	LangChain / LangGraph	LlamaIndex	PydanticAI
Primary role	Agent / application framework	Data / context framework	Type, schema, contract layer
Best at	Multi-agent orchestration, stateful workflows	RAG, document ingestion, retrieval	Structured output, type safety, testability
Orchestration	LangGraph: DAG with checkpointing	Event-driven async Workflows	Plain Python — no DSL
Type safety	Partial — inconsistent hints	Partial — Pydantic on events only	End-to-end — errors at write time

Dimension	LangChain / LangGraph	LlamaIndex	PydanticAI
State / memory	Built-in checkpointing (SQLite, Postgres)	Must build yourself	No built-in persistence
RAG	Possible, more boilerplate	Best-in-class	Not built-in — pair with vector store
Testing	Moderate — LangSmith helps	Moderate	Strong — DI enables zero-API-call tests
Observability	LangSmith (closed-source, paid)	LlamaCloud traces, OTEL supported	Logfire — OTEL-native, zero config
JS/TS support	Yes — LangChain.js, LangGraph.js	Partial — LlamaIndex.TS	Python only
V1 stable since	LangGraph 1.0 — Oct 2025	Workflows 1.0 — Jun 2025	V1 — Sep 2025

Common Failure Modes

Framework	Failure mode	Mitigation
LangChain	Abstraction leakage — custom behaviour requires fighting framework internals	Prefer LangGraph primitives over high-level chains for anything non-trivial
LangChain	Version churn — code breaks silently between minor LangGraph versions	Pin versions; allocate upgrade budget per release
LlamaIndex	Retrieval plateau — naive vector search degrades on large corpora	Invest in chunking strategy; use hierarchical retrieval
LlamaIndex	Context window truncation — default limits silently cut long documents	Set token limits explicitly; log chunk sizes
PydanticAI	Schema rigidity — overly strict schemas cause excessive retries	Start permissive; tighten schema after observing real LLM output
PydanticAI	Retry storm — auto-retry on validation failure amplifies costs	Always set <code>retries=</code> ; monitor <code>ctx.usage.input_tokens + output_tokens</code> # <code>RunUsage</code> fields in <code>result_validators</code>
PydanticAI	No built-in orchestration — complex multi-agent flows need LangGraph	Design agent boundaries first; add LangGraph when state management is needed
All three	LLM non-determinism — same prompt, different output across runs	Use <code>temperature=0</code> for extraction; add <code>result_validators</code> for business rules

Production Stack Patterns

Pattern	When to use it
PydanticAI only	Greenfield production agents, API services, type-safe pipelines with no RAG requirement
LlamaIndex → PydanticAI	Document intelligence — LlamaIndex handles ingestion and retrieval, PydanticAI handles structured output format
LangGraph + PydanticAI	Complex stateful multi-agent workflows — LangGraph provides checkpointing and graph structure, PydanticAI enforces I/O contracts at each node
All three	Full-stack agent platform — LlamaIndex (data), LangGraph (orchestration), PydanticAI (I/O contracts), Pydantic models as lingua franca throughout

Appendix B — LangGraph: Checkpointing and Fault-Tolerant Pipelines

LangGraph is a graph orchestration framework from LangChain that adds durable checkpointing, per-node retry policies, and conditional fallback routing to multi-agent pipelines. It is the answer to a problem the main paper identifies but does not fully solve: every failed pipeline run burned tokens on work that had already completed. LangGraph makes partial progress permanent.

In terms of the stack described in Appendix A.9: LangGraph is the orchestration layer; PydanticAI is the I/O contract layer at each node. LangGraph routes data between nodes and saves state. PydanticAI enforces typed schemas on every agent input and output. The two complement each other cleanly.

B.1 Why Checkpointing Matters — The Wasted-Work Problem

The paper's original pipeline ran INGEST → COLLECT → ENRICH → PRE-FETCH → REASON → REPORT → TRANSFORM sequentially in Python. When a connection reset hit REASON (the final LLM stage), the entire run failed. COLLECT, ENRICH, and PRE-FETCH — which consume zero API sessions — had to be re-executed from scratch on every retry. For pipelines that do expensive I/O work (API calls, database queries, file parsing) in their Python stages, this is a significant hidden cost.

 *LangGraph checkpoints the graph state at every node boundary. If REASON fails, the next run resumes from REASON — COLLECT, ENRICH, and PRE-FETCH are not re-executed. Partial progress is permanent.*

B.2 Per-Node Retry Policy — Selective Transient Error Handling

LangGraph also provides a RetryPolicy that can be attached to individual nodes. Combined with a selective retry predicate, this handles genuine transient errors (TCP drops, brief overloads) at the node level without retrying logical failures.

```
# pip install langgraph langgraph-checkpoint-sqlite

import httpx
```

```

from langgraph.graph import RetryPolicy

def should_retry(error: Exception) -> bool:
    """Only retry genuine transient errors.

    IMPORTANT: httpx.ReadError from the Anthropic API may be
    volume-induced
    rate limiting, not a genuine network transient. Use Logfire to
    measure
    your session count before deciding whether to retry connection
    resets.
    """
    return isinstance(error, (
        httpx.ConnectError,      # TCP connection refused
        httpx.ReadError,         # connection reset by peer
        httpx.TimeoutException,  # read/connect timeout
        ConnectionError,         # OS-level connection failure
    ))

# Attach retry policy only to LLM nodes (Python nodes are fast to re-run)
INGEST_RETRY = RetryPolicy(max_attempts=3, retry_on=should_retry)
REASON_RETRY = RetryPolicy(max_attempts=2, retry_on=should_retry)

```

B.3 Pipeline as a LangGraph — PydanticAI Agents as Nodes

Each pipeline stage becomes a LangGraph node function. LLM stages wrap a PydanticAI agent call. Python stages run subprocess commands directly. The graph topology matches the pipeline described in the main paper.

```

from typing import TypedDict, Any
from langgraph.graph import StateGraph, START, END
from pydantic_ai.exceptions import UnexpectedModelBehavior

# LangGraph state — equivalent to PipelineState dataclass in main paper.
# TypedDict is required; LangGraph merges partial state dicts between
nodes.
class PipelineState(TypedDict, total=False):
    record_ids: list[str]
    user_id: str
    ingest_results: Any
    collect_results: Any
    enrich_results: Any
    kb_data: Any
    ref_data: Any
    status_data: Any
    reason_results: Any

# — Node functions —————

async def ingest_node(state: PipelineState) -> dict:
    """LLM stage — wraps PydanticAI agent. ~3 API sessions."""
    agent = make_ingest_agent()          # fresh factory instance
    result = await agent.run(
        build_ingest_prompt(state),
        deps=state,
    )

```

```

    )
    return {'ingest_results': result.output}    # partial state update

async def collect_node(state: PipelineState) -> dict:
    """Python stage – direct subprocess, 0 API sessions."""
    results = await asyncio.gather(
        run_tool('probe_a.py', 'fetch', *state['record_ids']),
        run_tool('probe_b.py', 'fetch', *state['record_ids']),
    )
    return {'collect_results': results}

async def prefetch_node(state: PipelineState) -> dict:
    """Python stage – gather all data REASON needs. 0 API sessions."""
    kb, ref, status = await asyncio.gather(
        run_tool('knowledge_base.py', 'lookup', *state['record_ids']),
        run_tool('reference_lookup.py', 'query', *state['record_ids']),
        run_tool('status_check.py', 'get', *state['record_ids']),
    )
    return {'kb_data': kb, 'ref_data': ref, 'status_data': status}

async def reason_node(state: PipelineState) -> dict:
    """LLM stage – no tools, exactly 1 API session."""
    agent = make_reason_agent()                # fresh factory instance
    result = await agent.run(build_reason_prompt(state))
    return {'reason_results': result.output}

# — Build the graph —————

builder = StateGraph(PipelineState)

# Add nodes – retry policy only on LLM nodes
builder.add_node('ingest',    ingest_node,    retry=INGEST_RETRY)
builder.add_node('collect',  collect_node)
builder.add_node('enrich',   enrich_node)
builder.add_node('prefetch', prefetch_node)
builder.add_node('reason',   reason_node,     retry=REASON_RETRY)
builder.add_node('report',   report_node)
builder.add_node('transform', transform_node)

# Linear pipeline – edges connect stages in order
builder.add_edge(START,      'ingest')
builder.add_edge('ingest',   'collect')
builder.add_edge('collect',  'enrich')
builder.add_edge('enrich',   'prefetch')
builder.add_edge('prefetch', 'reason')
builder.add_edge('reason',   'report')
builder.add_edge('report',   'transform')
builder.add_edge('transform', END)

```

B.4 Checkpointing — Durable Resume After Failure

```

from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver
# Production: use PostgresSaver from langgraph-checkpoint-postgres

async def run_with_checkpointing(

```

```

    record_ids: list[str],
    user_id: str,
    thread_id: str,
) -> PipelineState:
    """Run the pipeline with durable checkpointing.

    If this run fails mid-way (e.g., connection reset at 'reason'),
    call this function again with the SAME thread_id.
    LangGraph resumes from 'reason'. COLLECT, ENRICH, and PRE-FETCH
    are NOT re-executed — their results are restored from the checkpoint.
    """

    async with AsyncSqliteSaver.from_conn_string('checkpoints.db') as
ckpt:
        graph = builder.compile(checkpointer=ckpt)
        config = {'configurable': {'thread_id': thread_id}}
        result = await graph.ainvoke(
            {'record_ids': record_ids, 'user_id': user_id},
            config=config,
        )
        return result

# First attempt (fails at 'reason' due to connection reset):
# await run_with_checkpointing(['rec_1'], 'u_1', 'run_2026_04_15_001')
#
# Checkpoint state after failure:
# ✓ ingest    → saved
# ✓ collect   → saved
# ✓ enrich    → saved
# ✓ prefetch  → saved
# ✗ reason    → failed here
#
# Second attempt with same thread_id — resumes from 'reason':
# await run_with_checkpointing(['rec_1'], 'u_1', 'run_2026_04_15_001')
# INGEST, COLLECT, ENRICH, PRE-FETCH: skipped (restored from checkpoint)
# REASON: executed fresh — succeeds

```

Checkpoint backends

Backend	Install	When to use
AsyncSqliteSaver	langgraph-checkpoint-sqlite	Local development, single-process, no persistence across restarts needed
AsyncPostgresSaver	langgraph-checkpoint-postgres	Production — survives process restarts, works across workers, queryable
MemorySaver	built-in	Testing only — in-memory, no durability

LlamaIndex Workflows — Per-step retry (note active bug)

LlamaIndex Workflows provide a similar per-step retry mechanism via a decorator:

```

from llama_index.core.workflow import Workflow, step, StartEvent,
StopEvent

```

```

from llama_index.core.workflow.retry_policy import
ConstantDelayRetryPolicy

class IngestWorkflow(Workflow):
    @step(retry_policy=ConstantDelayRetryPolicy(delay=5,
maximum_attempts=3))
    async def ingest(self, ev: StartEvent) -> StopEvent:
        # Rate limits and transient errors retry automatically
        ...

```

⚠ WARNING — Issue #20403 (December 2025): The LlamaIndex Workflow retry policy is currently broken. The delay parameter is ignored and the attempt counter resets after each failure, causing infinite immediate retries rather than bounded retries with backoff. Verify this is fixed at github.com/run-llama/llama_index before using retry_policy in production.

References

Resource	Description	URL
PydanticAI Documentation	Full API reference, agent patterns, changelog	https://ai.pydantic.dev
PydanticAI: HTTP Retries	AsyncTenacityTransport, wait_retry_after, AnthropicProvider integration	https://ai.pydantic.dev/retries/
PydanticAI GitHub	Source code, issues, release notes, version history	https://github.com/pydantic/pydantic-ai
Anthropic API: Errors	HTTP error codes, error shapes, request ID, long request guidance	https://docs.anthropic.com/en/api/errors
Anthropic API: Rate Limits	Tokens-per-minute, requests-per-minute, service tiers	https://docs.anthropic.com/en/api/rate-limits
Anthropic API: Streaming	SSE streaming, get_final_message(), error handling mid-stream	https://docs.anthropic.com/en/build-with-claude/streaming
Anthropic Message Batches API	Async batch processing; poll for results; tolerates network drops	https://docs.anthropic.com/en/api/creating-message-batches
Pydantic Documentation	BaseModel, Field, field_validator, model_validator reference	https://docs.pydantic.dev
LangGraph Documentation	Checkpointing, RetryPolicy, fault-tolerant graph execution	https://langchain-ai.github.io/langgraph
LangGraph Persistence	AsyncSqliteSaver, AsyncPostgresSaver, thread-based resume	https://langchain-ai.github.io/langgraph/concepts/persistence/
Logfire Documentation	OTel setup, instrument_httpx, metric_histogram, SQL queries	https://logfire.pydantic.dev/docs/
LlamaIndex Workflows: Retry	ConstantDelayRetryPolicy reference — verify issue #20403 is fixed	https://developers.llamaindex.ai/python/llamaagents/workflows/retry_steps/

Resource	Description	URL
FastAPI Documentation	BackgroundTasks, Depends, lifespan, async routes	https://fastapi.tiangolo.com
Anthropic Status Page	Live service status, incident history, uptime metrics	https://status.anthropic.com